Cinema Ticket Booking System

Assignment #4 — Facade Pattern

Student: Dias Sabit Language: Java 17 IDE: IntelliJ IDEA

GitHub: https://github.com/diassabit1/cinema-ticket-booking-facade

1. Introduction

A cinema booking operation requires several steps: locating a movie, checking a seat, reserving that seat, processing a simulated payment, and creating a ticket. If the client performs every step itself, it must also handle the order of operations and errors. This project uses the Facade structural design pattern to provide one simple entry point for that workflow.

2. Implementation and responsibilities

CinemaBookingFacade provides showAvailableMovies(), bookTicket(movieTitle, seatNumber, balance), and cancelBooking(ticketId). The Main client creates only CinemaBookingFacade. MovieService looks up films; SeatService validates and reserves places; PaymentService checks whether the supplied balance is enough; TicketService issues ticket IDs and keeps ticket records. Movie stores the title, duration and ticket price.

During booking, the facade finds a movie, validates and reserves the seat, checks payment, and generates a ticket. If payment fails, the facade releases the reserved seat. The payment step is a simulation: it validates a balance and does not debit a real account. Reservations are kept in memory; restarting the application resets them.

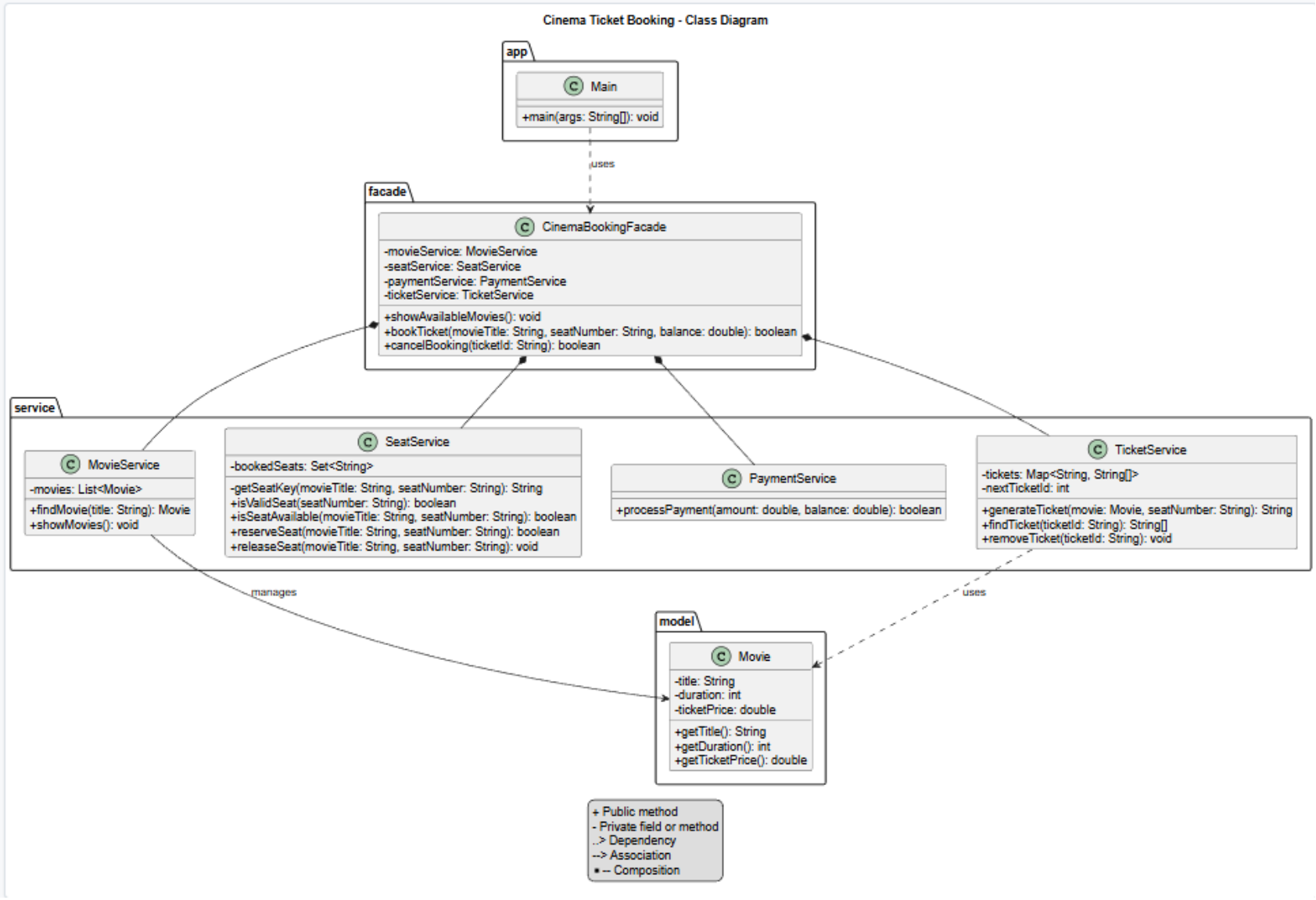
Design boundary: subsystem classes do not refer back to the facade. Their public methods remain available for advanced clients, even though the included Main client uses only the facade.

3. Test scenarios demonstrated

The console demonstration covers successful booking and ticket generation; duplicate-seat rejection; insufficient-balance rollback; the same seat on different movies; missing movie and ticket IDs; cancellation and rebooking of the released seat; and invalid seat syntax. These ten scenarios produced the expected messages during the user-reported run.

4. UML Class Diagram

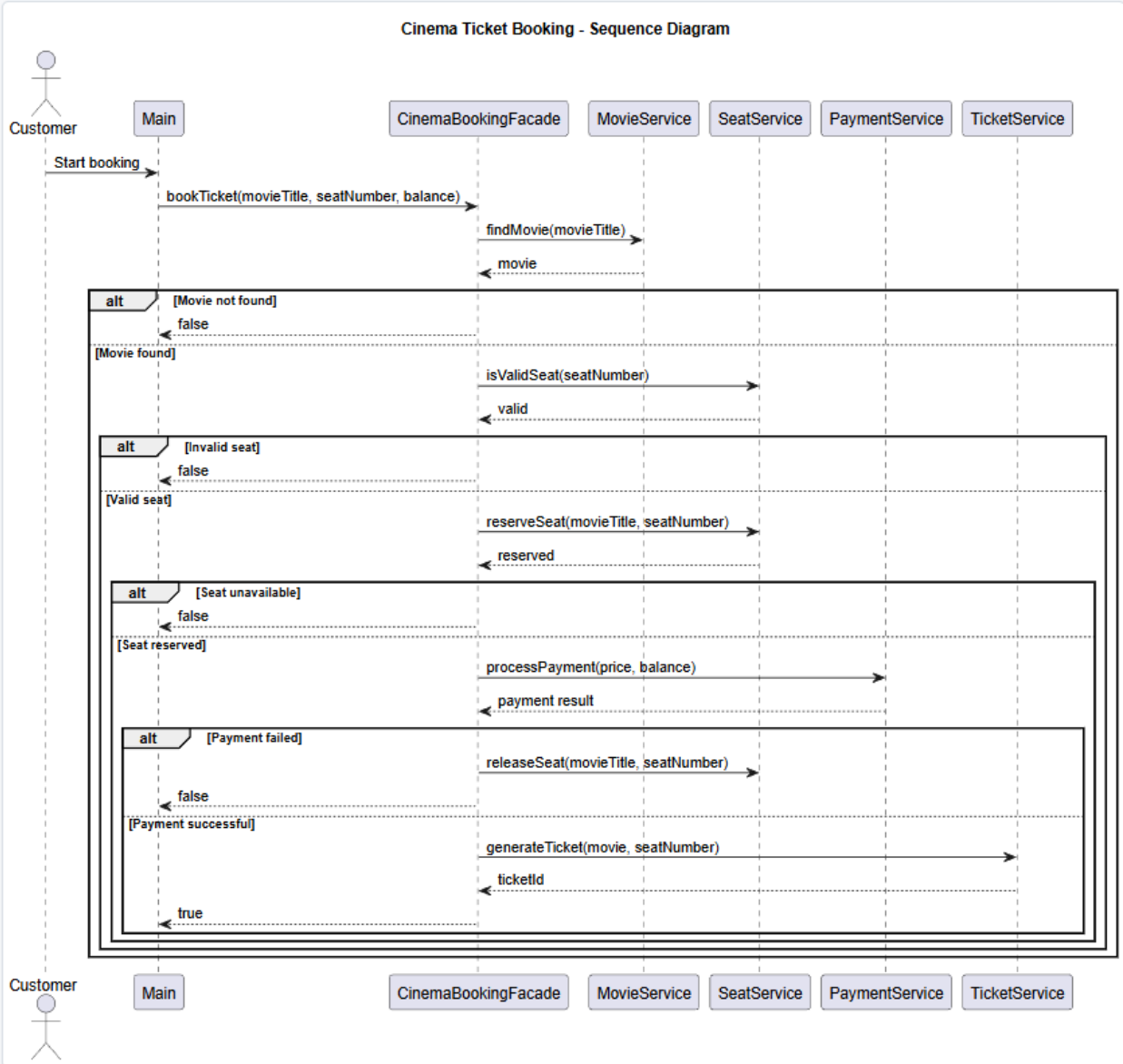
The class diagram shows the Main client, the facade, four subsystem services, and the Movie model. The facade owns the subsystem instances it creates. The client depends on the facade rather than calling individual services.



Notation: + = public member; - = private member; dashed arrow = dependency; solid association arrow = relationship; filled diamond = composition.

5. UML Sequence Diagram (additional)

This supplementary diagram shows the order of booking operations and the alternate branches for missing movies, invalid or unavailable seats, failed payments, and successful ticket generation.



6. Clean Code principles (1–3)

The following short before/after examples contrast a less clear approach with the organization used by the project. “Before” samples are illustrative, not claims about an earlier committed implementation.

1. Meaningful names

```
Before:  boolean doIt(String a, String b, double c)
After:   boolean bookTicket(String movieTitle, String seatNumber, double balance)
```

The public method and parameter names explain what the operation requires without opening its implementation.

2. Single Responsibility Principle

```
Before:  Main finds movies, reserves seats, checks payments, prints tickets.
After:   MovieService.findMovie(title)
         SeatService.reserveSeat(movieTitle, seatNumber)
         PaymentService.processPayment(amount, balance)
         TicketService.generateTicket(movie, seatNumber)
```

Each service handles one type of task. CinemaBookingFacade coordinates these services instead of duplicating their logic.

3. Hide subsystem orchestration from the client

```
Before:  client calls movieService, seatService, paymentService, ticketService
After:   CinemaBookingFacade booking = new CinemaBookingFacade();
         booking.bookTicket("Inception", "A5", 20.0);
```

The Main client only uses the facade. Internal service types do not appear as parameters or return values of bookTicket().

7. Clean Code principles (4–5)

4. Small, focused methods and reusable validation

```
Before:  repeat seat format checks in several booking methods
After:   public boolean isValidSeat(String seatNumber) {
             return seatNumber != null &&
                    seatNumber.matches("[A-Z][1-9][0-9]?");
         }
```

The rule for seat input is kept in one method of SeatService. The facade uses that method before attempting a reservation.

5. Fail-fast handling and rollback

```
Before:  keep the seat reserved even if payment is rejected
After:   if (!paymentService.processPayment(movie.getTicketPrice(), balance)) {
             seatService.releaseSeat(movie.getTitle(), seatNumber);
             System.out.println("Payment failed. Seat reservation cancelled.");
             return false;
         }
```

The facade stops when an operation cannot continue. A declined simulated payment reverses the earlier seat reservation, avoiding a seat being blocked without a ticket.

8. Limitations and possible extensions

The current project models seat availability separately for each movie title, but not for each date and showtime. It stores data only while the program is running and payment processing is simulated. An improved version could introduce a Screening model, persisted booking records, more detailed payment results, and automated unit tests.

9. Conclusion

CinemaBookingFacade makes the client code shorter and keeps service coordination in one place. It hides the multi-step booking sequence, including validation, rollback and ticket creation. Individual services are still public and directly usable by advanced clients. The design demonstrates the core intent of the Facade pattern with a manageable Java 17 example.

10. Repository

https://github.com/diassabit1/cinema-ticket-booking-facade

Topic note: Cinema Ticket Booking is a free-topic (Option C) implementation; the assignment requires instructor confirmation of a free topic.